ThreadLearn: A RAG-Augmented Fine-Tuned Language Model for JavaScript Concurrency Bug Detection and Fix Suggestion

Le Tri Trung[1] [0009-0007-0790-1388], Ha Van An[2] [0009-0000-8536-4396], and Nguyen Thi Thuy Hoai[3] [0009-0007-6328-0715]

[1] FPT University, Da Nang, Vietnam — letritrung2605@gmail.com

[2] FPT University, Da Nang, Vietnam — van.an.webdev@gmail.com

[3] FPT University, Da Nang, Vietnam — hoaintt40@fe.edu.vn

Abstract.

We introduce ThreadLearn, a system that detects JavaScript concurrency bugs and generates fix-pattern suggestions by combining a static race detector with a fine-tuned small language model. Existing tools either detect bugs without fixing them (ESLint, ThreadSanitizer) or require 7B–32B-parameter models that cannot be deployed on-premise. ThreadLearn fine-tunes Qwen2.5-Coder-1.5B with QLoRA on 892 hindsight Chain-of-Thought examples and augments inference with BM25 retrieval over a 2,050-document knowledge base. On a 30-case real-world benchmark from production npm packages spanning 10 bug categories, the full pipeline scores 22.0/30 (73.3%) versus 65.0% for GPT-3.5-turbo with the same pipeline, using a much smaller open-weight 1.5B model. A key finding is that the retrieval pipeline amplifies fine-tuning (+10 pp) but does not benefit models without domain knowledge (+0 pp for the base model).

Keywords: Concurrency bugs · Race conditions · Static analysis · LLM fine-tuning · Retrieval-augmented generation · JavaScript.

1 Introduction

Asynchronous JavaScript (Node.js [13]) powers web backends, APIs, and real-time services, but its event-driven, non-blocking model invites subtle concurrency bugs. A large-scale empirical study (NodeCB [1]) found that 93% of 57 real Node.js concurrency bugs cause crashes, corruption, or hangs. Despite how common and harmful these bugs are, existing tools still fall short. Rule-based static analyzers (ESLint async, ThreadSanitizer) detect suspicious patterns but cannot fix them, and they generate false alarms on modern async/await code. LLM-based approaches [2] reason about code semantics but cannot pinpoint the buggy line without structural guidance and require large models for competitive accuracy.

Four gaps drive this work: (G1) rule-based detectors find bugs but cannot generate fix suggestions; (G2) LLM-only approaches ignore static-structure signals and need 7B–32B models; (G3) no existing tool feeds detector output as structured context to an LLM; (G4) no tool covers detect→explain→fix-suggestion end-to-end for JavaScript concurrency bugs.

We structure the evaluation around three research questions:

- **RQ1.** Does combining a static race detector with a fine-tuned small language model outperform a general-purpose LLM (GPT-3.5-turbo) on real-world JavaScript concurrency bugs? (Sect. 5)
- **RQ2.** Does the full static-analysis and retrieval-augmented pipeline improve fix-suggestion quality equally across models, or does its benefit depend on prior domain fine-tuning? (Sect. 5)
- **RQ3.** Which concurrency bug categories remain hard for a fine-tuned small model, and what does that reveal about the limits of pattern-based training data? (Sect. 5)

The remainder of this paper is organized as follows. Section 2 provides background. Section 3 surveys related work. Section 4 describes dataset construction, static detector design, fine-tuning procedure, and end-to-end pipeline. Section 5 reports experimental results. Section 6 discusses implications. Section 7 concludes.

2 Background

Node.js concurrency bugs arise from the single-threaded event loop: one thread interleaves callbacks, I/O completions, and timers in scheduling-dependent order. NodeCB [1] reports three recurring types: atomicity violations (65%), order violations (30%), and starvation (5%). We add closure-loop-var—where function-scoped var lets deferred callbacks observe the final loop value—fixable by var→let. BM25 [6] retrieves by term frequency and IDF (k1=1.5, b=0.75); exact API-name matching (e.g., setTimeout) makes sparse retrieval more precise than dense methods for concurrency patterns. QLoRA [3] fine-tunes a 4-bit quantized base model via LoRA [10] adapters (r=16, α=32), keeping training within a single GPU.

3 Related Work

3.1 Static and dynamic bug detectors.

ThreadSanitizer [4], ESLint [11], NodeCB [1], and NACD [5] detect bugs via runtime instrumentation, syntactic rules, regex matching, or delay injection—but none generates fix suggestions (G1). ThreadLearn adds fix-suggestion capability on top of static detection.

3.2 LLMs for code repair.

PCWMs [2] applies reasoning world models to parallel code but requires 7B–32B parameters and ignores static structure. ThreadLearn uses a 1.5B model guided by detector output, addressing G2 and G3.

3.3 RAG and fine-tuning for code.

RAG [7] improves LLM grounding for knowledge-intensive tasks; QLoRA [3] and LoRA [10] enable domain SFT on consumer hardware. Neither approach has been applied jointly to JavaScript concurrency remediation. ThreadLearn combines both with hindsight CoT [16], filling this gap.

Table 1. ThreadLearn vs. related tools. ✓ = supported, ✗ = not supported.

Tool	Detect	Fix Suggestion	JS	Fine-tuned
ThreadSanitizer	✗	✗	✗	✗
ESLint async	✓	✗	✓	✗
NodeCB (rules)	✓	✗	✓	✗
PCWMs (LLM-only)	✓	✓	✓	✓
ThreadLearn (This work)	✓	✓	✓	✓

Rule-based tools flag a line without suggesting a fix; PCWMs generates suggestions but without structural signal, requiring 7B–32B parameters. ThreadLearn combines all four capabilities: its detector narrows search to pattern_id and line_range before LLM inference, letting a 1.5B model perform competitively on this benchmark (Sect. 5).

4 Proposed Approach

4.1 Dataset

Our study uses three datasets. The fine-tuning dataset (892 examples) has the form (code, detector_output, reasoning_trace, fix). Fix labels are human-verified by the authors with no crowdsourcing; reasoning traces were produced with teacher-model assistance and manually reviewed to ensure correctness. It comprises 332 synthetic pairs (37.2%)—handcrafted and manually verified—and 560 generated pairs (62.8%) produced by generate_pairs(), a template engine that expands the handcrafted pairs over 40 domain slots (e.g., User, Order) using 18 generator functions, each a distinct concurrency transformation (e.g., sequential await→Promise.all, callback→async, unguarded fetch→AbortController timeout). Because each generator is a deterministic template, all generated labels are correct by construction, avoiding the label noise of LLM-sampled data.

The knowledge base contains 2,050 JavaScript documents (MDN [15], concurrency-library docs, curated pattern descriptions, and synthetic permutations).

The real-world benchmark has 30 JavaScript concurrency bugs from production open-source packages, spanning 10 categories: Race Condition (5), Unhandled Rejection (5), Double Callback (4), Resource Exhaustion (4), Event Loop Blocking (3), Sequential Awaits (3), Zalgo (2), Context Loss (2), Stream Leak (1), and Callback Hell (1). Each case traces to a specific GitHub issue, eliminating model-favorable bias.

4.2 Architecture and Static Detector

Four esprima-based [9] preprocessing helpers canonicalize source before detection: stripComments and normalizeWhitespace remove formatting noise; extractFunctions scopes line-ranges to function bodies; extract_keywords returns the top-20 identifiers as the BM25 query.

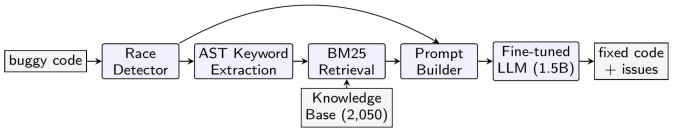

Fig. 1. ThreadLearn end-to-end pipeline: buggy code → static detection → AST keyword extraction → BM25 retrieval → prompt construction → LLM inference → fix suggestion.

Pipeline.

The pipeline (Fig. 1) runs five steps: (1) static detection—race_detector scans the source in O(n) time, emitting {pattern_id, line_range, description} per match; (2) keyword extraction—extract_keywords() walks the esprima AST, keeping CamelCase API names intact; (3) retrieval—BM25 returns the top-3 of 2,050 documents; (4) prompt construction—prompt_builder assembles detector report, retrieved docs, and buggy code; (5) fix generation—the fine-tuned LLM outputs corrected code with reasoning.

Static race detector.

The detector checks 5 patterns in two groups. Closure/async: closure_loop_var (var captured by reference in loop callbacks), shared_var_settimeout (shared variable modified inside setTimeout), promise_no_await (Promise created but not awaited). Shared state: concurrent_write_array (array mutated from concurrent callbacks) and counter_no_atomic (non-atomic counter increment). For example, closure_loop_var fires when a var loop variable is captured inside a setTimeout or Promise callback; the fix replaces var with let.

Detector coverage across benchmark categories.

Table 2 maps the 5 detector patterns against the 10 benchmark categories and shows the level of direct support each category receives. Categories without a matching detector

pattern (Indirect) rely on BM25-retrieved context to guide the LLM toward the correct fix class; the static detector's output is still included in the prompt as structural context.

Table 2. Benchmark categories by detector support level.

Benchmark Category	Detector Pattern	Support
Race Condition	counter_no_atomic, concurrent_write_array	Direct
Unhandled Rejection	promise_no_await	Direct
Sequential Awaits	promise_no_await	Direct
Closure Loop Var	closure_loop_var	Direct
Shared Var setTimeout	shared_var_settimeout	Direct
Double Callback	callback guard heuristic	Partial
Resource Exhaustion	resource-loop heuristic	Partial
Event Loop Blocking	sync-call heuristic	Partial
Zalgo / Context Loss / Stream Leak / Callback Hell	none	Indirect (RAG only)

Five categories have Direct detector support; three Partial (heuristic); four rely solely on RAG context (Indirect). This structure predicts the per-category accuracy in Sect. 5.3.

5 Experimental Results

All experiments use the 30-case benchmark (Sect. 4.1), run on a Kaggle T4 GPU for local models and the OpenAI API for GPT-3.5-turbo [14].

5.1 Evaluation Metrics

We evaluate six configurations—{base, fine-tuned, GPT-3.5-turbo} × {no-pipeline, +pipeline}—using the following scoring scheme. Each test case specifies an expected fix pattern (e.g., Promise.all, try/catch, let in loop):

- **PASS (1.0)** — generated output contains the expected fix pattern and is syntactically valid code.
- **PARTIAL (0.5)** — syntactically valid code generated, but without the expected fix pattern.
- **FAIL (0)** — no code generated.

We acknowledge that PASS/PARTIAL/FAIL is a proxy metric: it rewards identifying the correct fix class rather than guaranteeing behavioral correctness. A model could score PASS while introducing a semantically different change. Validating behavioral correctness via automated unit test pass rates is left for future work.

5.2 Main Results

Fig. 2. Full ablation on the 30-case real-world benchmark (score = PASS + 0.5×PARTIAL).

Configuration	Pass	Partial	Fail	Score	%

Base (no fine-tune, no pipeline)	6	24	0	18.0	60.0%
GPT-3.5-turbo (no pipeline)	7	23	0	18.5	61.7%
Fine-tuned only (no pipeline)	8	22	0	19.0	63.3%
Base + pipeline	8	20	2	18.0	60.0%
GPT-3.5-turbo + pipeline	9	21	0	19.5	65.0%
ThreadLearn + pipeline	**14**	**16**	**0**	**22.0**	**73.3%**

Fine-tuning contribution (no pipeline): +1.0 pt (+3.3 pp). Pipeline contribution (fine-tuned): +3.0 pt (+10.0 pp). Combined gain vs. base: +4.0 pt (+13.3 pp).

Without any pipeline, the three models score in a narrow band (60.0–63.3%) with zero FAILs: every model emits syntactically valid code, and the dominant failure mode is surface-level rewriting into async/await without fixing the underlying hazard. Adding the full pipeline then separates the models sharply: the fine-tuned model gains +3.0 pts (+10 pp), GPT-3.5-turbo only +1.0 pt, and the base model nothing (60.0% in both configurations) while adding 2 FAILs.

5.3 Per-Category Analysis

Breaking 22.0/30 down by category, ThreadLearn is strongest on Sequential Awaits (100%), Unhandled Rejection (90%), and Race Condition (70%, up from 50% for both baselines)—categories tied to Direct detector support and specific fix templates. It is weakest on Zalgo, Double Callback, and Stream Leak (all 50%), the last being the only category where it trails the base model. Per Table 2, these are Indirect (RAG-only) or Partial categories. This answers RQ3: bugs requiring tracing callback invocation order across asynchronous boundaries—a semantic property not inferable from local syntactic context—remain hard because template-generated training data is dominated by syntactic substitutions.

5.4 Threats to Validity

The 30-case benchmark is small; differences of 2-3 cases are not statistically significant without formal testing (e.g., McNemar test). PASS/PARTIAL/FAIL is a proxy metric (Sect. 5.1) — behavioral correctness via automated test suites is future work.

6 Discussion

(i) Fine-tuning is a prerequisite for pipeline benefit. The base model cannot leverage retrieved documents—adding the full pipeline produces no improvement and introduces 2 additional FAILs. This confirms G2: static-structure signals and retrieved context are only useful when the model already has domain knowledge to apply them.

(ii) The pipeline amplifies fine-tuning, not the reverse. The pipeline adds +3.0 pts on top of fine-tuning's +1.0 pt—a 3× multiplier. This suggests the optimal deployment sequence is: fine-tune first, then augment with retrieval.

(iii) Competitive accuracy at smaller scale. On this 30-case preliminary benchmark, ThreadLearn (73.3%) shows a higher score than GPT-3.5-turbo with the same pipeline (65.0%), using a much smaller open-weight 1.5B model. We caution that the benchmark is small and this difference is not statistically validated (Sect. 5.4).

Answering RQ1 and RQ2.

RQ1: the static detector's structured output narrows what the LLM must infer, enabling a fine-tuned 1.5B model to show higher scores than GPT-3.5-turbo on this benchmark. RQ2: the full static-analysis and retrieval-augmented pipeline does not benefit every model equally—+10 pp for the fine-tuned model versus 0 pp (and regression) for the base model—so pipeline augmentation is only effective when paired with domain-specific fine-tuning. Note that the pipeline comprises AST keyword extraction, BM25 retrieval, static detector output, and augmented prompt construction; isolating the individual contribution of each component requires dedicated ablation beyond the scope of this paper.

Latency and deployment context.

ThreadLearn's inference latency (approximately 26 s per case on a T4 GPU) is higher than GPT-3.5-turbo API calls (approximately 1.8 s). This positions ThreadLearn for offline code review and IDE-assisted repair workflows, not real-time production monitoring. The trade-off is data privacy: ThreadLearn runs entirely on-premise without sending proprietary source code to external API infrastructure, avoiding risks of data leakage, IP exposure, or compliance violations (GDPR, HIPAA, corporate policies). For enterprises handling sensitive codebases, this privacy advantage complements the accuracy result.

7 Conclusion and Research Directions

ThreadLearn addresses the gap between static detection and automated remediation by combining a 5-pattern rule-based race detector with a small language model (Qwen2.5-Coder-1.5B) fine-tuned on 892 hindsight Chain-of-Thought examples and a BM25+AST retrieval pipeline. On a 30-case real-world benchmark from production npm packages, ThreadLearn achieves 22.0/30 (73.3%)—a +13.3 pp gain over the base model and +8.3 pp over GPT-3.5-turbo with the same pipeline, using a much smaller open-weight 1.5B model. These results are preliminary and should be interpreted with caution given the small benchmark size and unvalidated statistical significance (Sect. 5.4).

To the best of our knowledge, ThreadLearn is among the first attempts to pair a rule-based race detector with a fine-tuned small language model and RAG pipeline for JavaScript concurrency bug fix suggestion, bridging PCWMs [2] (LLM-only, large model) and rule-based detectors (ESLint, ThreadSanitizer) that produce no fix suggestions.

The ablation establishes a design principle: pipeline augmentation requires prior domain fine-tuning. Future work will extend the detector to TypeScript, validate fixes via automated test suites, and test larger fine-tuned models on the hard Indirect categories.

References

1. Wang, J., Dou, W., Gao, Y., Gao, C., Qin, F., Yin, K., Wei, J.: A comprehensive study on real world concurrency bugs in Node.js. In: Proc. 32nd IEEE/ACM ASE, pp. 520–531 (2017).
2. Singh, G., Guha, A., Kailkhura, B., Menon, H.: Learning reasoning world models for parallel code. arXiv:2604.20926 (2026).
3. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: QLoRA: efficient finetuning of quantized LLMs. In: Proc. NeurIPS (2023).
4. Serebryany, K., Iskhodzhanov, T.: ThreadSanitizer: data race detection in practice. In: Proc. WBIA, pp. 62–71 (2009).
5. Endo, A.T., Møller, A.: Event race detection for Node.js using delay injections. In: Proc. 39th ECOOP, pp. 9:1–9:28 (2025).
6. Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. Found. Trends Inf. Retr. 3(4), 333–389 (2009).
7. Lewis, P., Perez, E., Piktus, A., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Proc. NeurIPS (2020).
8. Qwen Team: Qwen2.5-Coder technical report. arXiv:2409.12186 (2024).
9. Hidayat, A.: Esprima: ECMAScript parsing infrastructure. https://esprima.org, last accessed 2026/06/30.
10. Hu, E.J., Shen, Y., Wallis, P., et al.: LoRA: low-rank adaptation of large language models. In: Proc. ICLR (2022).
11. Zakas, N.C.: ESLint: the pluggable JavaScript linter. https://eslint.org, last accessed 2026/06/30.
12. von Werra, L., Belkada, Y., Tunstall, L., et al.: TRL: transformer reinforcement learning. https://github.com/huggingface/trl, last accessed 2026/06/30.
13. Dahl, R.: Node.js: evented I/O for V8 JavaScript. https://nodejs.org, last accessed 2026/06/30.
14. OpenAI: GPT-3.5 Turbo. https://platform.openai.com/docs/models, last accessed 2026/06/30.
15. Mozilla: MDN Web Docs: JavaScript reference. https://developer.mozilla.org/en-US/docs/Web/JavaScript, last accessed 2026/06/30.
16. Wei, J., Wang, X., Schuurmans, D., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: Proc. NeurIPS (2022).
17. Brown, D.: rank-bm25: a collection of BM25 algorithms in Python. https://github.com/dorianbrown/rank_bm25, last accessed 2026/06/30.